

Version 3.41.0 · 2026-08-05 · Will Hooper

A slide-by-slide, jargon-light tour. The white paper (linked on the last slide) has the math and sources.

A family of small AI models for competitive Pokémon (Champions VGC). Built from thousands of real ladder games. Honest about what it can and can't know.

You **can't reliably predict who wins** a match from the two team sheets — in this format it's close to a coin flip, and even a strong player-rating model barely beats a coin. So ABRA stops trying to call the winner and instead **helps you make better decisions**: what to bring, and what to do turn by turn.

This is the same move sports analytics made with "expected goals" — you can't predict the final score, but you can measure the value of each shot. ABRA is that idea, pointed at Pokémon.

One rule: **keep every real game, and analyse on top of it**. ABRA saves every public ladder replay into a growing library and builds its models on that library — so it gets smarter as more games come in, and never has to re-download anything. It runs on a normal laptop; no special hardware.

Each model is a "house" you can visit on the site:

- **MEDICHAM** — the damage engine. Matches the community standard, but disagrees with the game's OFFICIAL engine by a wide margin, so it is being replaced by that engine and kept only as a lookup.
 - **MEW** — plays the official engine against itself, so we are not limited to the games people happen to upload.
 - **GURU** — reads the metagame: which team styles beat which, from real results, with error bars.
 - **XATU** — reads the opponent: the likely item, ability, and moves behind each Pokémon.
 - **PORY** — mid-battle win chance. **Retracted as a finding**: it works out to counting how many Pokémon each side has left and how healthy they are, and it does not beat exactly that baseline.
 - **The scoring bot** — the part that finally *looks at the board*.
 - **SLOWKING** — the strategist: there's no single best team, so it plays a smart mix.
 - **CHOMP** — the team picker: which four to bring, which two to lead.
 - **ALAKAZAM** — the coach (in progress): the best move to make, right now.
-

Everything ABRA claims about "this build beats that build" comes from playing the game out against a practice opponent. That opponent used to pick its move purely by **how popular the move is** — it had no idea what was standing in front of it. So it fired attacks at Pokémon that were flat immune, it used moves that couldn't possibly work, and when there were two enemies to aim at, it picked one by flipping a coin.

We fixed it by **watching what people actually do**. We took about 48,000 real decisions from games where both players' full teams were public — so we could see not just what someone clicked, but everything they *could* have clicked — and learned how much a real player's choice depends on what's in front of them. Nobody wrote the rules; they were measured.

The result, checked on games the learning never saw:

- it now hits the enemy's weakness **half again as often** as before,
- it wastes **a third fewer** moves on things that simply cannot work,
- and it almost never fires at something immune any more.

It is still short of a human on all three, and one thing it still does badly is decide **when to switch** — that's the next job. But the practice opponent is no longer playing blind, which means the numbers it produces are worth more than they were.

PORY also beats a coin flip mid-game, using how many Pokémon each side has left and their health, and it's *calibrated* — when it says 70% it's really about 70%. It's live on the site as a per-turn "you're at X%".

Two things we tested and reported straight, even though they're negatives:

- **Picking the team doesn't beat a coin.** We tested whether the team-picker's choices track who wins — they don't, more than chance. So we don't oversell it. The damage math is still exact and useful.
- **The "rock-paper-scissors" metagame is a hint, not a fact.** The data suggests Trick Room beats Hyper Offense beats Sand beats Trick Room — but each of those rests on only ~13–18 games, so the error bars are wide. We label it suggestive, and it'll firm up as more games arrive.

Added 3.32.0 — three more, and the first one is the worst thing we've found.

- **We thought Rage Powder was deleting attacks. It wasn't — and catching that is the story.** The test we wrote to check it aimed a *Dragon*-type attack at a Pokémon that is immune to Dragon. The redirection worked perfectly and moved the attack onto something that takes zero damage, so the test saw zero either way and we believed the wrong thing for several hours. Re-run against a Pokémon that can actually be hurt, the untouched code works fine. Nothing was broken and nothing we had measured was invalidated. **A test can be wrong in exactly the shape of the bug it is looking for** — that is now the ninth time here, and the first time it fooled us.
- **A real one next door, though.** Lightning Rod and Storm Drain — abilities that pull attacks in — genuinely weren't working, 1,901 uses. Fixed.
- **The number we quote for "how often we disagree with the real game" wasn't repeatable.** It picked its test matchups at random and never wrote down which ones. Two runs on identical code gave 6 and then 3. It now uses a fixed list, and the honest figure is 4 disagreements in 400.
- **The part of the bot that judges "am I winning?" is no better than a coin flip.** We had measured this before on 350 games — too few to support the claim either way. On 6,886 games it's decisive: when it says it's 94% to win, it wins 54%; when it says 6%, it also wins 54%. Worse, the version we had been measuring wasn't the one the bot actually uses.

Added 3.33.0 — we went looking for the same mistake everywhere else, and found it twice more.

The 3.32.0 note above says a result we'd published as a pass turned out to be unreproducible. Here's *why*, because the reason is more useful than the result: the file we kept recorded the answers and not the settings. It's like keeping a stopwatch time and not writing down which race it was. Two runs with very different settings produced files that were identical, and we could not tell which one we had.

So we checked the other three tests in the same series.

- **"The new search picks a different move 73% of the time" is a headline with its control missing.** A search that's just guessing disagrees with *everything* — including with itself. So the test measures that too: run the same search twice with different dice and see how often it contradicts itself. That number is the yardstick, and the test printed it to the screen and never saved it. On an early version the yardstick was *higher* than the headline. We believe the 73% is real, because the yardstick shrinks as you give the search more thinking time and this run had plenty — but believing and having measured are different things, and it's the second one we publish.

- **"A leaf costs 5.83 milliseconds" was timing something the bot doesn't do.** The measurement quietly used the settings the code falls back to when you don't say, and the bot uses different ones — a randomised payout at triple the length. Not wrong arithmetic; the wrong thing measured.

Every one of these tests now writes a small companion file recording exactly what it did: how many rollouts, what randomness, what horizon, which version of every file it read, and whether anything was uncommitted at the time. Older results get one reconstructed from the commit that carried them, labelled as a guess rather than a record.

Added 3.34.0 — the clearest example yet of how this goes wrong.

We built a page showing every model, published it, and asked for a review. It was completely broken — one stray quotation mark meant none of it could run, so it showed a title and a blank screen.

Two automated checks looked at that page and both gave it full marks. One counted twelve model cards and found twelve. The other checked that every number on it cited a source, and scored it 100%. Neither of them ever *ran* the page — they read it the way you'd read a document. So a page that could not work passed every test we had.

A person opening the link found it in about two seconds.

The same day, a second version of the same problem: two new pages were written, tested, saved and published to the code repository — and never copied to the folder the live website actually serves. The check meant to catch that compared one specific file, so a *missing* file was invisible to it. Again: found by opening the URL.

Both checks now exist in a form that would have caught them, and both were written the same way — by asking what the check could not see, rather than what it did see.

Saying what we *can't* prove, as plainly as what we can, is the whole point.

We also caught ourselves twice this week, which matters more than any single fix. A result we'd published as a pass — "playing the position out beats just counting what's left" — turned out to be unreproducible from anything we'd kept, and on the evidence that survives it's a tie, not a win. And a claim written in our own notes at 3am about an outside dataset was wrong within four hours; it's corrected in place with the reason, not quietly deleted.

The one lesson that changed the plan. We spent a day teaching the bot more about Pokémon — four separate things it didn't know before. All four made no difference at all. Then we changed *what it was trying to do*, twice, and both times it got a lot better.

Teaching it more facts had stopped helping. Changing its goal was what worked.

The reason is simple. The bot was trained to **copy what humans click**. That is a different goal from **winning**. It even shows up in the numbers: the model rates "use a spread move that hits both of them and doesn't hurt my own partner" as a *bad* idea — not because it is bad, but because humans rarely click it. A bot told to avoid its own best plays will avoid them.

- **Teach the two Pokémon to act as a team.** We already built this once. It plays worse, and now we know why: it was taught to *resemble* people rather than to *win*. Retraining it on the right goal is the single most promising thing on the list.
- **Fix a small unfairness in the training data.** When a Pokémon is locked into one move, we were still showing the bot all four as if it had a choice. It affects about 1 in 15 items.
- **Find out how readable it is.** A rival bot built only to beat ours won 63% of the time. That was measured a long time ago and never repeated. A bot can get better on average while staying just as easy to

read — those are different questions, and only one of them has been checked lately.

- **Then, looking ahead.** Right now the bot only thinks about *this* turn. Looking one turn further is a real project, but it's now measured rather than guessed: there are only **72** sensible combinations to consider each turn, not the trillions people assume.
- **ALAKAZAM** — the in-battle coach. A light version runs in your browser; a version that could beat top humans needs a rented cloud computer and months of the AI playing itself.

Something we got wrong, kept here on purpose. For two days nobody updated these documents while the code kept changing. The next person to read them — which was us — then described several models completely incorrectly, including calling one "not built yet" when it had been built, tested, and measured. Documents that lag the work don't just go quiet; they actively mislead.

Four teams work on ABRA at once. That is deliberate — it is why the work was split four ways.

One night all four were running. Their files were kept separate so nobody could overwrite anybody. A 7,100-game experiment — "can anything beat our bot?" — was destroyed anyway.

The reason is simple once you see it. Halfway through the experiment, another team **improved the bot being tested**. So the first half of the experiment played against the old bot and the second half played against the new one. The final number described neither.

Nothing broke. No error appeared. The result looked completely normal, and the automatic checker approved it — because that checker was asking "*is this file newer than the thing it came from?*" and the answer was yes. A file can be newer than something it never actually read.

The fix is not to make the teams take turns. That would throw away the whole reason for having four of them.

Instead, an experiment now works from a **photograph of the bot** rather than the bot itself. Before measuring, we freeze a copy of everything that could change the answer, and the experiment reads the frozen copy. Other teams can rewrite the real thing all night; the experiment never notices.

The honest consequence: **we currently have no answer to "how beatable is our bot?"** The old answer was measured on a version of the bot that no longer exists, and the new attempt is void. Saying so is better than quoting a number we cannot stand behind — that number had been called the most important one in the project.

One thing did survive, and it is genuinely useful: when the bot plays a mirror match against itself it wins 49.7% of the time, which is a coin flip. That confirms our testing setup does not quietly favour one side — a worry an earlier, smaller sample had raised.

We found something bigger than the bug we went looking for.

In Champions, players often reveal their whole team before the battle. Our bot **uses** that information when it plays. But when we *taught* the bot, we were throwing half of it away — the bot learned without knowing the opponent's abilities or moves, then plays with both.

It affects **half of every decision** it was trained on, and virtually every game. It is like training a driver in fog and then handing them the keys on a clear day: not obviously worse, but they never learned to use what they can now see.

We have not fixed it yet, and that is deliberate. Fixing it means retraining everything, and there is a real question first: sometimes opponents *decline* to reveal their team. A bot trained to depend on information that is sometimes missing can fail badly when it disappears. We have made that mistake before, in exactly this shape, and it is worth one deliberate decision rather than a quick fix.

Update (version 3.40.0). The decision was made: the bot plays with the full team sheet, always, and the rarer no-sheet games are set aside for now. The first half of the retraining is done — the move-picking layer now learns from the same information it plays with, and we *counted* that the information actually arrived during training (99.7% of decisions) instead of assuming it. The second half (the layer that picks pairs of actions) is queued next. No claim yet that the retrained bot is better — that comparison is set up against a frozen copy of the old one and runs next.

Also in this pass: the simulator learned eight more rule families and now covers 181 of the 186 mechanics we probe for, with the remaining 5 named and explained; and we compared our two separate "rulebook" files fact by fact — they disagreed exactly twice, and one of those (how often Iron Head flinches in this format) was a real bug the bot had been playing with. It reads the right number now.

Update (version 3.41.0, same night). The second half of the retraining is done too — the layer that picks pairs of actions now learns from the full team sheet, and we measured what all that sheet information is actually worth by replaying 45,000 real decisions with and without it. Honest answer: the bot judges positions measurably better with the sheet, but we cannot yet show it clicking different moves often enough to prove a win-rate gain — the effect is smaller than our measurement noise. We say it exactly that way rather than rounding up. Meanwhile the simulator's coverage grew again (202 of 205 probed mechanics, 3 remaining, each with a written reason), the mirror-match test hit 100% agreement with the official engine, and two real bugs — one in how Intimidate interacts with certain abilities, one where Sheer Force was hurting its own holder — were found and fixed with proof.

Full technical detail, math, and sources: [ABRA white paper](#). Also: [project summary](#) · [technical docs](#) · [model ledger](#).